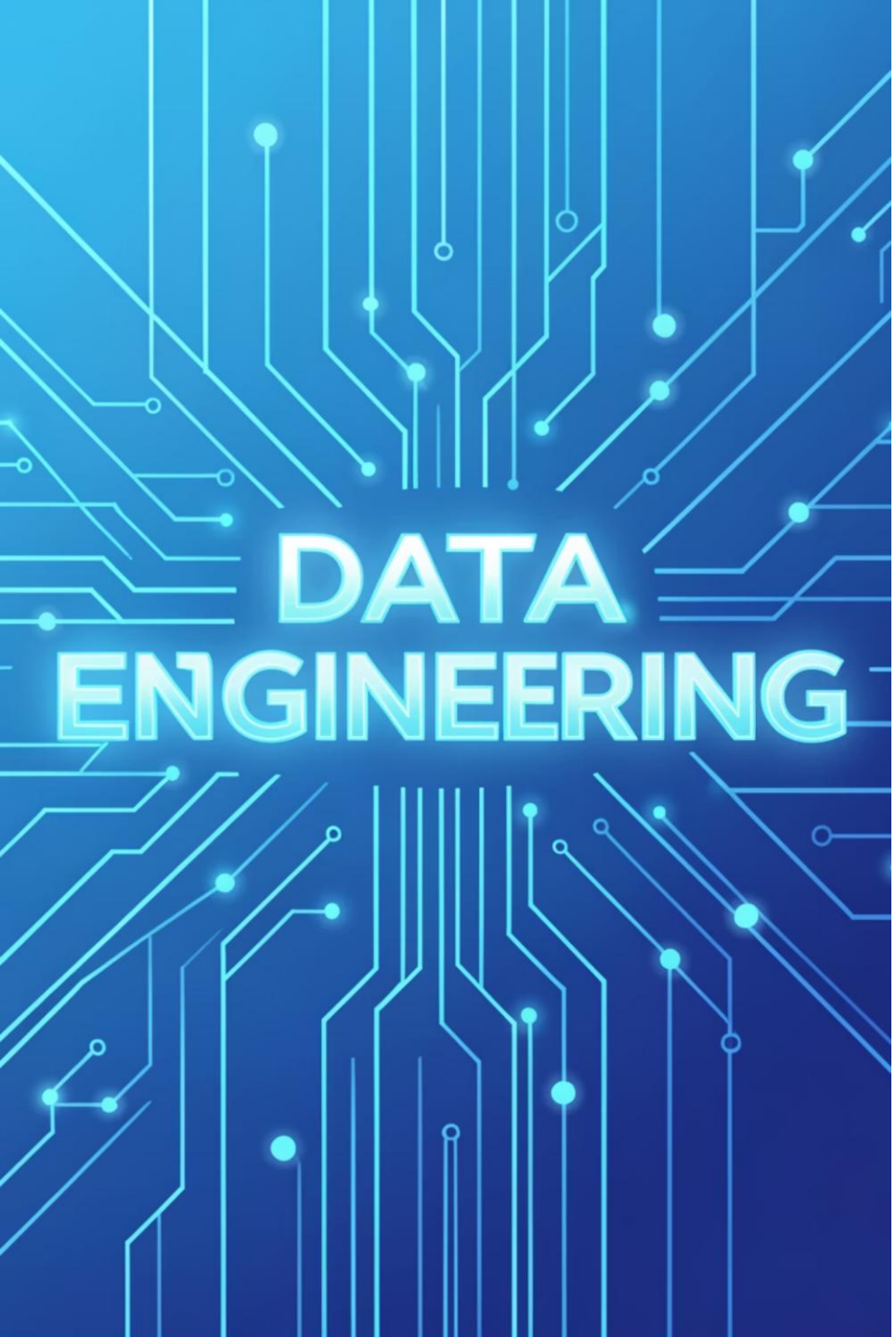

Data Engineering

Spring '26 – DS463

Mar 04, 2026

Lecture 21

Logistics: Your Course Essentials Essentials

Class Timings

Monday (LH6), Tuesday (LH8): 11:30 AM – 12:20 PM

Friday (LH8): 11:00 AM – 11:50 AM

Credit Hours

2 hours (Lectures) + 1 hour (Practical (Practical Session)

Office Hours

Monday, Tuesday: 10:00 AM – 11:00 AM, AM, S18 2nd Floor NAB or by appointment.

Contact & Support

MS Teams: w5t93yk

Email: farah.saeed@giki.edu.pk

WhatsApp: Reach out for quick queries

Source systems: Practical Details

Databases

- Common source system database.
- There are as many types of databases as there are use cases for data.

Source systems: Practical Details

Databases : Relational Database Management Systems (RDBMS)

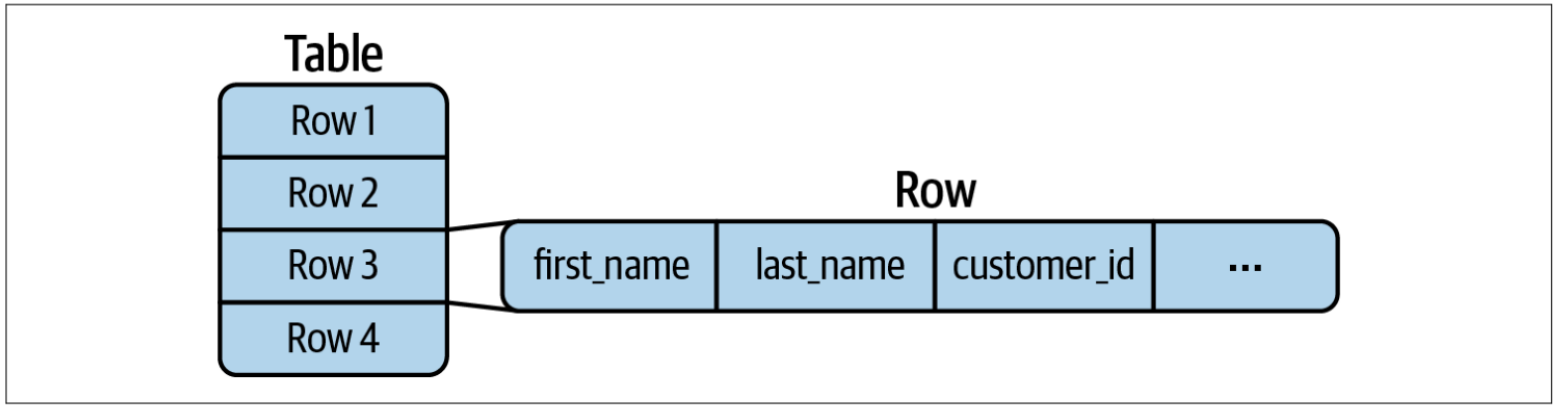

Figure 5-7. RDBMS stores and retrieves data at a row level

Source systems: Practical Details

Databases : Relational Database Management Systems (RDBMS)

- Data is stored in tables.
- Rows (also called relations)
- Columns (also called fields)
- Each row represents one record.
- Follows the same schema (same column structure).
- Each column has a fixed data type (e.g., string, integer, float).

Primary Key

- A unique identifier for each row.
- No two rows can have the same primary key.
- Used for indexing and fast lookup.

Foreign Key

- A field that links to the primary key of another table.
- Joins
- Complex schemas

Source systems: Practical Details

Databases : Relational Database Management Systems (RDBMS)

Normalization

- A database design strategy.
- Ensures data is not duplicated across tables.
- Stored efficiently.
- Example:Instead of repeating customer address in many tables,store it once and reference it via foreign key.

Most RDBMS follow ACID Principles

Atomicity – All-or-nothing transactions

Consistency – Database remains valid

Isolation – Transactions don’t interfere

Durability – Committed data is not lost

Source systems: Practical Details

Databases : Non relational Databases (NoSQL)

NoSQL stands for "Not Only SQL."

It refers to databases that:

- Do not follow the relational model
- Do not rely on tables with strict schemas

Designed for:

- High scalability
- Flexible data
- Large-scale distributed systems

Advantages:

- Better horizontal scalability
- More flexible schemas
- Improved performance for specific use cases

Trade-offs:

- Often weaker consistency guarantees
- Limited or no joins
- No fixed schema
- May not be fully ACID-compliant

Source systems: Practical Details

Databases : Key Value Store

Key-Value Store

Example: Web app session storage

When a user logs into Daraz:

Key → user_12345 Value → {loggedIn, “Product A“}

The app quickly retrieves session data using the user ID.

Used for:

- Login sessions
- Shopping carts
- Caching frequently accessed data

Key:

user_101

Value:

"John, Premium User, 3 orders"

Source systems: Practical Details

Databases : Document Store

Document Store : Document JSON like

Example: User profile in a social media app

The app quickly retrieves session data using the user ID.

Used for:

- User profiles
- Product catalogs
- Blog platforms
- Mobile app backends

```
{
  "user_id": 123,
  "name": “abc",
  "email": “abc@email.com",
  "posts": [
    {"id": 1, "content": "Hello"},
    {"id": 2, "content": "Vacation time"}
  ]
}
```

Source systems: Practical Details

Databases : Document Store

Document Store : Document JSON like

Example: User profile in a social media app

The app quickly retrieves session data using the user ID.

Used for:

- User profiles
- Product catalogs
- Blog platforms
- Mobile app backends

```
{
  "user_id": 123,
  "name": “abc",
  "email": “abc@email.com",
  "posts": [
    {"id": 1, "content": "Hello"},
    {"id": 2, "content": "Vacation time"}
  ]
}
```

Table 5-2. Comparison of RDBMS and document terminology

RDBMS	Document database
Table	Collection
Row	Document, items, entity

Source systems: Practical Details

Databases : Wide column database

Wide column database

Example: Ecommerce order tracking

Used for:

- Large ecommerce systems
- Ad tech tracking
- IoT data at scale

```
{
  "user_id": 123,
  "name": “abc",
  "email": “abc@email.com",
  "posts": [
    {"id": 1, "content": "Hello"},
    {"id": 2, "content": "Vacation time"}
  ]
}
```

Source systems: Practical Details

Databases : Graph database

Graph database

Example: Social Media connections

Nodes: Users

Edges: "connected_to“

Query example:

"How many 2nd-degree connections does John have?“

"Suggest new connections based on mutual connections."Graph

Used for:

- Social networks
- Fraud detection (who is connected to suspicious accounts?)
- Recommendation engine

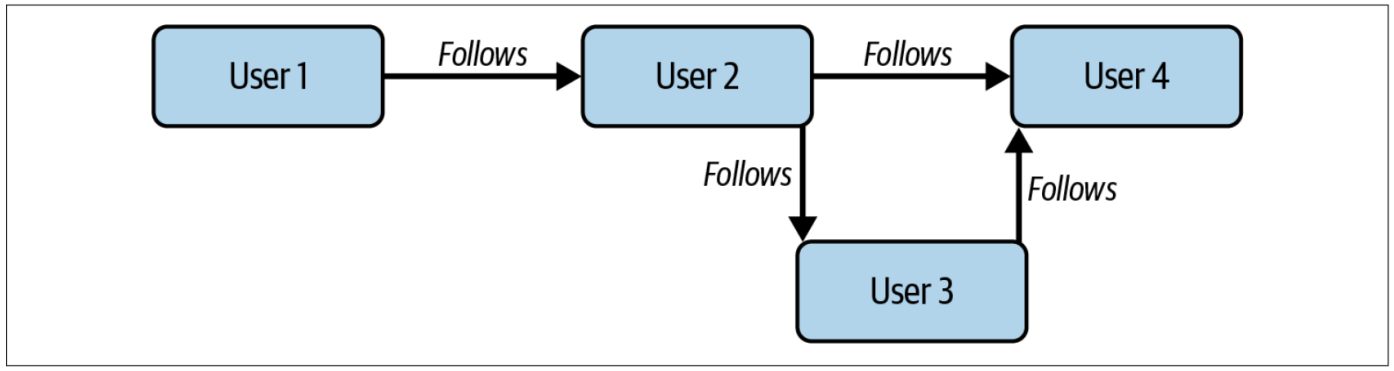

Figure 5-8. A social network graph

Source systems: Practical Details

Databases : Search database

Search database

Example: Product search

User types:"wireless noise cancelling headphones“

Search database:

Matches exact words

Matches similar spellings

Ranks by relevance

Used for:

- Log analysis ("Find all errors in last 5 minutes")
- Monitoring systems
- Security event search

Source systems: Practical Details

Databases : Timeseries database

Time series database

Example: Weather monitoring system

Used for:

Stock prices

Application metrics

Server monitoring

Used for:

- timestamp: 10:01 → temperature: 72°F
- timestamp: 10:02 → temperature: 73°F
- timestamp: 10:03 → temperature: 72°F

Source systems: Practical Details

Databases : Timeseries database

Time series database

Example: Weather monitoring system

Used for:

Stock prices

Application metrics

Server monitoring

Used for:

- timestamp: 10:01 → temperature: 72°F
- timestamp: 10:02 → temperature: 73°F
- timestamp: 10:03 → temperature: 72°F

Source systems: Practical Details

Application Programming Interface APIs

APIs

APIs (Application Programming Interfaces)APIs are a standard way for systems to exchange data.

Widely used in:

- Cloud services
- Internal company systems

Most modern APIs use HTTP (same protocol used by websites).

Source systems: Practical Details

Application Programming Interface APIs

Representational State transfer REST API

REST APIs

REST is the most common API style today.

Uses HTTP methods like:

GET → retrieve data

PUT → update data(also commonly POST, DELETE)

Stateless Each request is independent.

The server does not remember previous requests.

Source systems: Practical Details

Application Programming Interface APIs

GraphQL

GraphQL

Designed as an alternative to REST APIs.

It is a query language for APIs.

Advantage:

- You can request multiple types of data in one single query.
- More flexible than REST.
- You choose exactly what data you want.

Data Format

- Built around JSON.
- Returns data in a structure similar to your query.

Source systems: Practical Details

Application Programming Interface APIs

GraphQL

GraphQL Example

Query Request

GraphQL Query (Request)

```
query {
  user(id: "123") {
    name
    email
    posts(limit: 2) {
      title
      createdAt
    }
  }
}
```

JSON Response


```
{
"data": {
"user": {
"name": "Ali Khan",
"email": "ali@example.com",
"posts": [
{
"title": "My First Post",
"createdAt": "2026-02-20"
},
{
"title": "GraphQL Basics",
"createdAt": "2026-02-18"
} ] } } }
```

Source systems: Practical Details

Application Programming Interface APIs

Webhooks

A simple event-based data transmission method.
Triggered when a specific event happens.

An event happens in the source system (app, backend, website, etc.).
The source system automatically sends data to a specific HTTP endpoint.
This endpoint belongs to the data consumer.

Difference
Normal APIs: We request data.
Webhooks: The source pushes data automatically.
Often called reverse APIs.

Impact of undercurrents on source systems

Security

Data Protection: Make sure data is	Network Access: Check	Protect Credentials	Trust but Verify
• Encrypted at rest (when stored) • Encrypted in transit (when being sent over a network)	• Are you accessing the system over the public internet? • Or through a more secure VPN (Virtual Private Network)?	• Keep sensitive information secure through • Passwords • API tokens • SSH keys	• Make sure the source system is legitimate. • Verify that data is coming from a trusted source. • Avoid accidentally ingesting data from malicious actors.

Impact of undercurrents on source systems

Data Management

Data Governance

- Is the data properly managed and documented?
- Is it clear who owns the data?

Data Quality

- How accurate and reliable is the upstream data?
- Are there checks to ensure: there arent missing values and duplicates. Ensuring correct formats.

Schema Changes

- Expect that table structures (schemas) change.
- Plan for new columns, data type change

Master Data Management (MDM)

- Are records (e.g., customers, products) centrally controlled?

Privacy and Ethics

- Do you get raw data or masked/obfuscated data?
- Think about ethical implications of using the data.

Regulatory Compliance

- Are you legally allowed to access and use this data?
- Consider regulations (e.g., industry rules, data protection laws).
- Ensure compliance before ingestion.

Impact of undercurrents on source systems

Data Ops

Automation

Source System Automation

- Updates, new features, or code changes in the source system.
- These changes might break data pipeline.

Observability

- How will you know if the source system goes down?
- How will you detect bad data?

Set up:

- Monitoring for system uptime.
- Alerts

Incident response

- What happens if the source system goes offline?
- Does your pipeline fail, retry, or pause?
- How will you recover missing data?

Impact of undercurrents on source systems

Source system Architecture

Reliability	Durability	Availability	People & Ownership
• Does the system produce consistent outputs? • Predictable and stable performance.	• How is data protected from loss? • Prevent data loss.	• System is up and running when required.	• Who owns and maintains the source system?

Source systems: Practical Details

Orchestration and Source Systems

When orchestrating data workflows, we ensure:

- Proper network access
- Correct authentication
- Proper authorization

Orchestration system must securely connect to the source system.

Cadence and frequency

Is data available:

- On a fixed schedule (e.g., daily batch)
- Anytime(on-demand or real-time)

Align pipeline schedule with data release timing.

Data Engineering Lifecycle

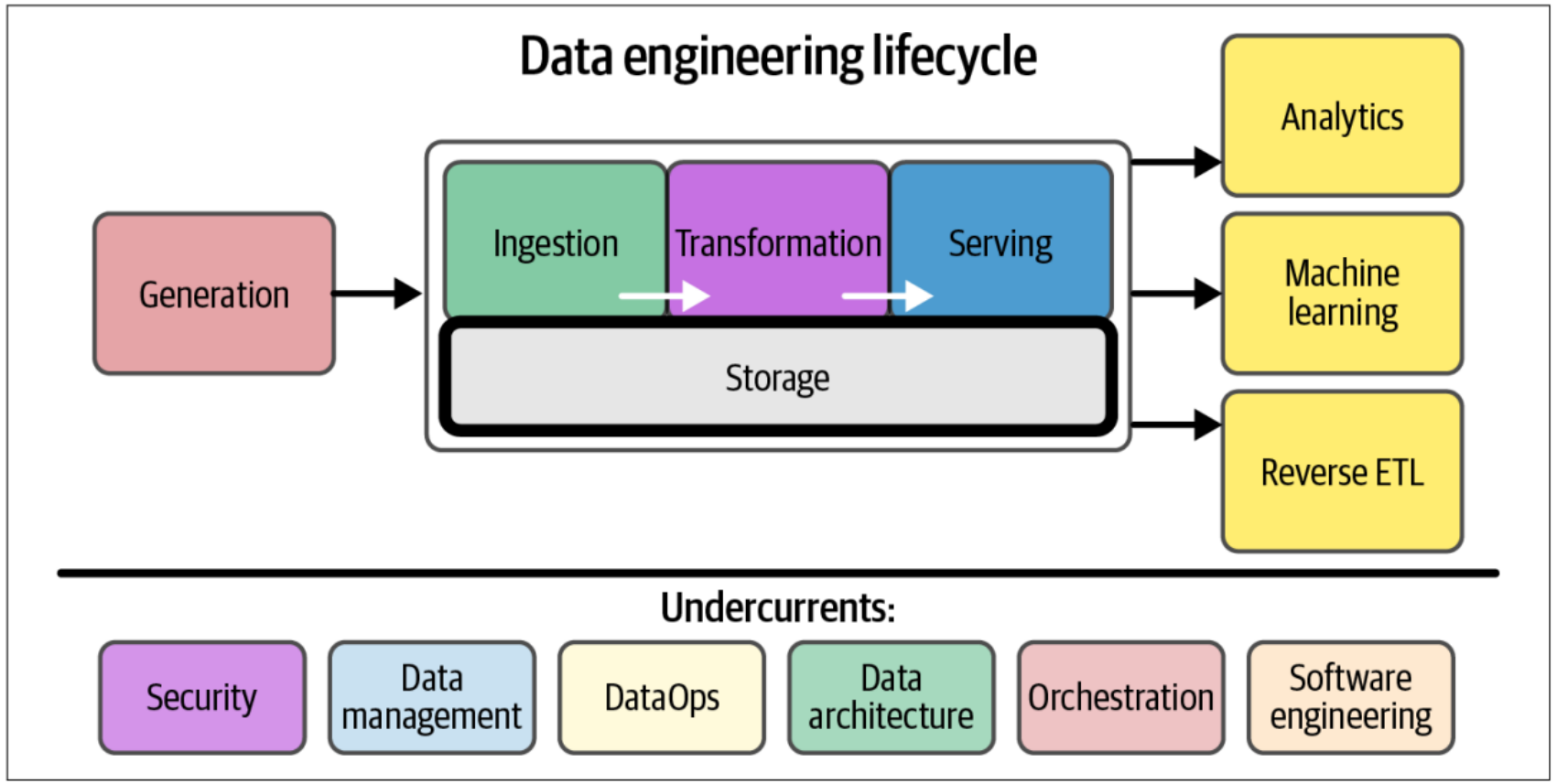

Figure 6-1. Storage plays a central role in the data engineering lifecycle

Storage is the cornerstone of the data engineering lifecycle underlies its major stages—ingestion, transformation, and serving.

Storage

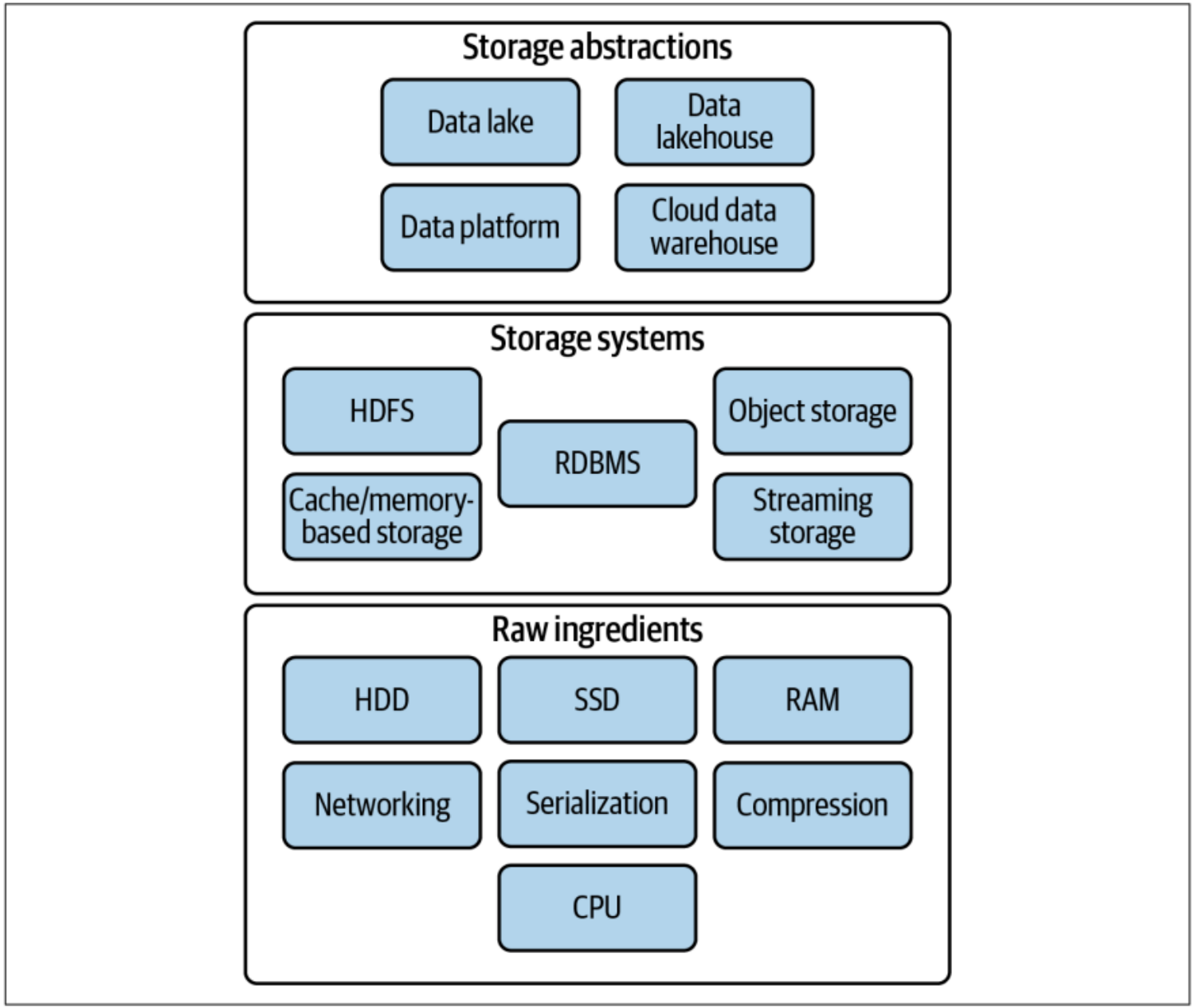

Figure 6-2. Raw ingredients, storage systems, and storage abstractions

Storage

Storage Type	Speed	Cost	Persistence
CPU Cache	Fastest	Very high	Volatile
RAM	Very fast	High	Volatile
SSD	Fast	Medium	Non-volatile
HDD	Slow	Cheap	Non-volatile